SSH HONEYPOT USING MACHINE LEARNING INTEGRATION

ABSTRACT

This project introduces an advanced **SSH honeypot** designed to simulate a legitimate server environment, effectively attracting and monitoring **unauthorized access attempts**. The system convincingly imitates real SSH functionalities, encouraging attackers to interact with it. Every **command**, **credential**, and **system interaction** from the attacker is thoroughly logged for detailed analysis.

The collected data is processed using sophisticated **machine learning (ML) techniques** to detect **threat patterns**, identify **anomalies**, and analyse **attacker behaviour**. This analysis helps recognize **attack trends**, uncover new vulnerabilities, and detect **malicious activities** in real time.

By studying these interactions, the system can anticipate potential threats and enhance **cybersecurity defences**. The integration of ML allows the honeypot to continuously learn and adapt to evolving attack strategies, ensuring proactive threat detection. This project provides valuable insights for strengthening security infrastructures, improving **threat intelligence**, and effectively combating modern **cyber-attacks**.

Keywords: **SSH Honeypot**, **Unauthorized Access Attempts** ,**Command Logging** ,**Credential Capture System Interaction Machine Learning (ML)** ,**Threat Detection** ,**Anomaly Detection** ,**Attacker Behaviour Analysis** ,**Attack Trends**, **Malicious Activities**, **Cybersecurity Defences**, **Threat Intelligence**, **Proactive Threat Detection** ,**Cyber-Attacks**, **Vulnerability Analysis** ,**Security Infrastructure** ,**Data Analysis** ,**Real-time Monitoring** ,**Adaptive Security System**

1. INTRODUCTION

Cybersecurity and threat detection have become crucial in the modern digital landscape as cyberattacks continue to evolve in complexity and frequency. Organizations must implement robust security measures to protect their systems, networks, and sensitive data from malicious actors who exploit vulnerabilities to gain unauthorized access. One of the most commonly targeted protocols is Secure Shell (SSH), which enables remote system access.Attackers frequently attempt to exploit weak SSH credentials, misconfigurations, or software vulnerabilities to take control of a system. Traditional security mechanisms, such as firewalls and intrusion detection systems, offer some protection but often struggle to detect sophisticated or evolving attack techniques.

This limitation underscores the necessity of advanced security strategies, such as deception-based security and machine learning-driven threat detection. Deceptive security systems, like SSH honeypots, are instrumental in understanding attacker behaviour. These systems mimic real servers, tricking attackers into engaging with them and revealing their strategies, tools, and techniques. By monitoring and logging these interactions, security teams can analyse attack patterns and develop more effective countermeasures. However, manually analysing large volumes of attack data can be time-consuming and inefficient. This is where machine learning plays a critical role in threat detection. Unlike traditional rule-based security systems, machine learning continuously learns from new data, adapting to emerging threats and improving its predictive capabilities.

This dynamic approach reduces response times, minimizes false positives, and strengthens overall cybersecurity resilience. By integrating machine learning with cybersecurity, organizations can proactively detect, analyse, and mitigate threats, ensuring a more adaptive and robust security posture. Ultimately, the combination of deception technology and artificial intelligence empowers organizations to stay ahead of cybercriminals and fortify their defences against evolving cyber threats.

2. REVIEW OF RELATED WORKS

Warren Z. Cabral, Craig Valli, Leslie F. Sikos, and Samuel G. Wakeling authored the paper **"Advanced Cowrie Configuration to Increase Honeypot Deceptiveness". Cowrie** is a medium-interaction SSH and Telnet honeypot that logs brute-force attacks and SSH requests. However, its publicly available code makes it vulnerable to detection. Most deployments use default configurations, making them easier for attackers to identify using tools like NMAP and Shodan. This research proposes a customizable framework to enhance Cowrie's deception and realism, comparing default and modified setups to evaluate their effectiveness.

Sajeel Mehta, Dipti Pawade, Yash Nayyar, and Ashwini Dalvi wrote the paper **"Cowrie Honeypot Data Analysis and Predicting the Directory Traverser Pattern of Attack"**. The **Cowrie Honeypot** is used to **entrap intruders**, log their activities, and analyse their behaviour to enhance **security**. Logged data is processed and visualized using **ELK (Elasticsearch, Logstash, Kibana)**, helping identify **attack patterns**. Additionally, a **probabilistic approach** predicts **intruder directory traversal patterns**, enabling proactive security measures.

Vasu Sethia and **A. Jeyasekar** authored the paper **"Malware Capturing and Analysis using Dionaea Honeypot"**. The rapid growth of the internet has led to increased **cyber threats**, including **malware**, **spyware**, and **botnets** that steal sensitive data and cause harm. This study deploys a **honeypot** to capture **zero-day attacks** and analyse **network logs** to classify **malware** based on behaviour. These insights help researchers develop **security mechanisms** to protect organizations from **malicious activities**.

Melike Baser, Ebu Yusuf Guven, and Muhammed Ali Aydın authored the paper **"SSH and Telnet Protocols Attack Analysis Using Honeypot Technique: Analysis of SSH and Telnet Honeypot"**. Traditional **cybersecurity measures** often fail against **zero-day attacks**, making **honeypots** essential for recording **attacker behaviour** and developing new **defence strategies**. This study deploys an **SSH-Telnet honeypot** using **Cowrie** to log attacker activities and analyse threats. Collected data, including logs and uploaded files, is shared on **GitHub** to aid further research.

Christoph Pohl, Alf Zugenmaier, Michael Meier, and Hans-Joachim Hof wrote the paper **"B. Hive: A Zero Configuration Forms Honeypot for Productive Web Applications"**. **Honeypots** detect intrusions by attracting attackers and logging their actions for analysis, aiming to remain **undetected** while appearing **valuable**. This paper introduces **Hive**, a novel honeypot that protects **active web applications** while concealing its presence from detection. **Hive** requires **no configuration changes**, is **web framework agnostic**, and has **minimal impact on performance**, making it effective for protecting **top websites**.

---

3. METHODOLOGY AND PROPOSED SYSTEMS

3.1 METHODOLOGY

• Dionaea Honeypot for Malware Capture Emulates vulnerable network services like SMB, FTP, HTTP, and MySQL to attract attackers.

• Logs network traffic, collects malware samples, and records attacker interactions.

• Supports automated malware submission for deeper cybersecurity analysis.

• Helps in identifying new attack vectors and studying malware behaviour. Cowrie Honeypot for SSH & Telnet Attack Monitoring

• Simulates a real SSH/Telnet server to capture brute-force attack attempts.

• Logs credentials, executed commands, and session activities for analysis.

• Integrates with ELK (Elasticsearch, Logstash, Kibana) for real-time monitoring.

• Enhances deception by allowing attackers to believe they have accessed a real system. Kippo Honeypot for SSH Attack Analysis

• Focuses on studying SSH brute-force attacks by allowing attackers to log in with weak credentials. • Captures login attempts, command execution, and session activities.

• Provides detailed logs to reconstruct attack scenarios for forensic analysis.

• Although outdated, its successor, Cowrie, builds upon its functionality with improved features. T-Pot: Multi-Honeypot Threat Intelligence System

• Combines multiple honeypots (Cowrie, Dionaea, Glastopf, Conpot) into a unified system.

• Monitors threats across various attack vectors, including SSH, web applications, and industrial systems.

• Offers built-in visualization tools like ELK for real-time data analysis.

• Used for large-scale cybersecurity research and enterprise-level threat monitoring. Honeyd for Network Emulation and Attack Logging

• Creates virtual hosts to mimic different operating systems and services.

• Simulates network vulnerabilities to study attack methodologies.

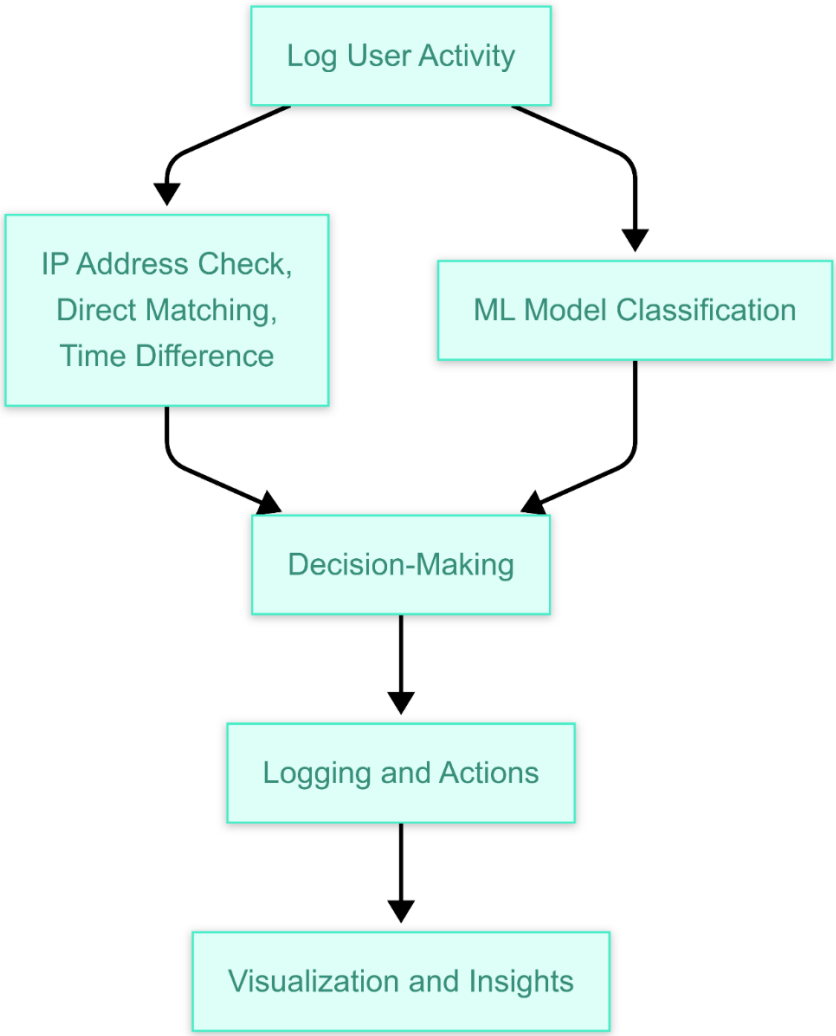
• Captures network scanning, fingerprinting, and unauthorized access attempts.

• Useful for detecting attacker behaviors and strengthening network security

---

PROPOSED SYSTEMS

3.2.1 Block Diagram



3.2.2 MODULES

1.SSH Honeypot Simulation This module sets up a deceptive SSH server using the Cowrie honeypot to attract attackers and log their activities. It records login attempts, executed commands, and uploaded files, providing valuable insights into attacker behaviour. The honeypot mimics a real SSH environment, ensuring high interaction levels while maintaining security and deception.

2.Intruder Activity Logging & Analysis All interactions with the SSH honeypot are logged in real-time, capturing credentials, session details, and executed commands. The collected logs are structured and preprocessed using log parsing algorithms. The processed data is then analysed for anomaly detection and behavioural pattern recognition.

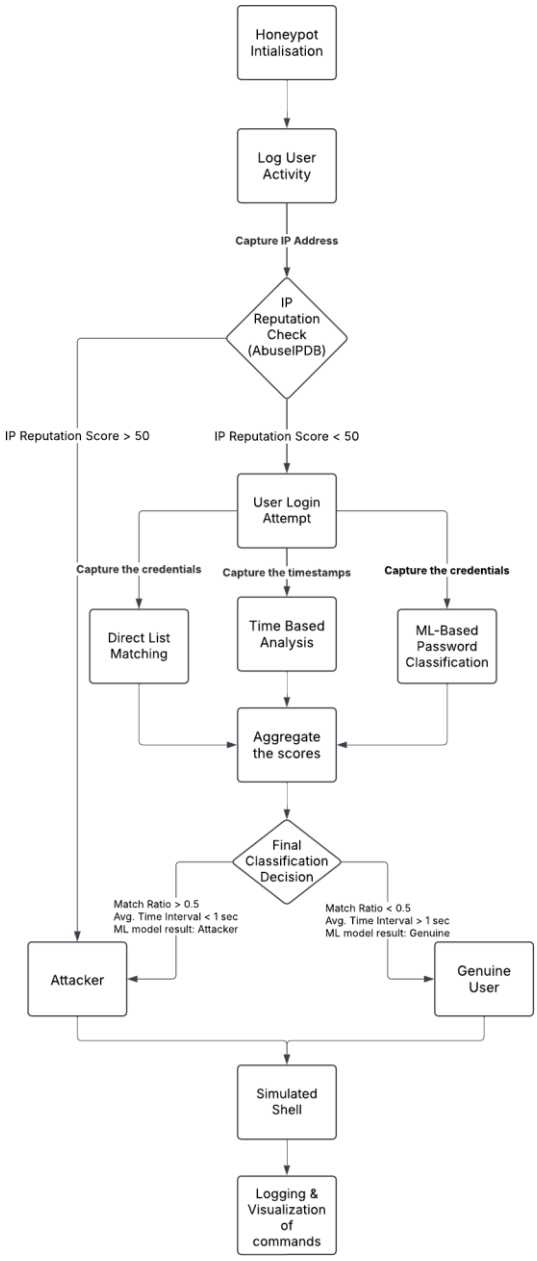
3.Machine Learning-Based Threat Detection This module integrates Isolation Forest and TF-IDF vectorization to detect unusual patterns in attacker behaviour. Machine learning techniques help classify login attempts as legitimate or malicious and identify frequently used attack commands. This enhances proactive threat detection and intrusion response.

4.Data Visualization & Reporting Attack trends and intruder behaviour are visualized using tools like ELK Stack, Matplotlib, and NetworkX. Graphical representations include command flow graphs, heatmaps of login attempts, and statistical summaries of attack patterns. This helps cybersecurity researchers and system administrators understand emerging threats and take preventive measures

5.Automated Alert System This module enhances real-time monitoring by sending alerts when suspicious activity is detected. Using Socket and Threading, it continuously monitors incoming connections and triggers alerts via email or Telegram APIs when predefined attack thresholds are met.

6.Custom Honeypot Configuration & Deployment To improve deception, this module allows security professionals to modify the SSH honeypot configuration dynamically. Unlike default Cowrie setups, it offers customizable environments that make detection harder for attackers using NMAP or Shodan scans. The honeypot can be deployed on virtual machines or cloud platforms for scalable cybersecurity research

3.2.3 **FLOW DIAGRAM**



---

4. DATASET AND ALGORITHM

4.1 DATASET

Our SSH honeypot project utilizes two primary datasets:

- ➢ Attack Log Dataset

➢ **Machine Learning Dataset**

4.1.1 Attack Log Dataset

This dataset captures real-time intrusion attempts, logging attacker behaviours and patterns. It helps analyse threats, detect anomalies, and improve cybersecurity defences.

• Intrusion Attempts: Logs failed and successful SSH login attempts.

• Command Execution History: Records all commands entered by attackers.

• Brute Force Data: Stores commonly attempted usernames and passwords.

• Malware Uploads: Logs files uploaded to the honeypot for further analysis.

• Geolocation Information: Tracks attacker IP addresses, ISPs, and countries.

• Time-Based Trends: Identifies peak attack hours and long-term attack trends.

4.1.2 Machine Learning Dataset

• This dataset is used to train models for detecting malicious activities and classifying attacks. • Attack Signatures: Categorizes login attempts as brute force, privilege escalation, or reconnaissance.

• Command Frequency Patterns: Identifies commonly used attack commands.

• Anomaly Scores: Uses machine learning models like Isolation Forest for anomaly detection. • Session Duration Metrics: Tracks how long an attacker interacts with the honeypot.

• Network Traffic Data: Captures SSH packets for deep behavioural analysis.

• Threat Intelligence Integration: Cross-references known malicious IPs and attack patterns from external sources.

4.2 ALGORITHMS

Log Parsing, Machine Learning, and Visualization Techniques Our SSH honeypot system processes raw intrusion logs, applies machine learning-based anomaly detection, and visualizes attack behaviours to gain deeper cybersecurity insights.

Below is a detailed breakdown of the key techniques used:

4.2.1. Log Parsing & Preprocessing Before performing analysis, raw SSH logs from the honeypot need to be structured and cleaned.

Log Parsing Algorithm:

• Extracts key fields such as timestamps, attacker IPs, usernames, attempted passwords, command sequences, and session durations.

• Filters out irrelevant system logs, keeping only malicious activity data for efficient processing.

Data Preprocessing Techniques:

• Label Encoding: Converts categorical data (e.g., usernames, attack types) into numerical form to facilitate machine learning analysis.

• Standardization: Uses Standard Scaler to normalize features (e.g., session duration, command frequency) to a mean of 0 and variance of 1, improving model stability.

4.2.2. Machine Learning-Based Anomaly Detection To identify malicious SSH activities, the system applies anomaly detection techniques:

Isolation Forest Algorithm:

• Trains on normal SSH command patterns to detect deviations in attacker behaviour.

• Flags suspicious login attempts, unknown commands, or privilege escalation attempts as outliers.

4.2.3. Supervised Learning (Optional):

• If labelled data is available, models like Random Forest, XGBoost, or SVM can be used to classify attack types (brute force, reconnaissance, or malware execution).

• Helps in predicting whether an attacker is a bot or a human based on behaviour pattern

4.2.4. Attack Pattern Recognition & Visualization

To better understand attacker strategies, the system visualizes data using text mining and graph-based techniques.

• TF-IDF (Term Frequency - Inverse Document Frequency):
• Analyses the most frequently used SSH commands by attackers.
• Identifies rare or unique commands executed by sophisticated hackers. Results are visualized using:
• Bar charts – Displays top commands used by attackers.
• Word clouds – Highlights most common commands visually.
• Histograms – Shows frequency distributions of attack attempts over time. Graph Theory (Network Analysis):
• Nodes represent individual commands, while edges represent transitions between them.
• Helps visualize how attackers navigate through the system, whether they are exploring directories, modifying files, or escalating privileges.
• Implemented using NetworkX and Matplotlib to generate attack flow graphs

---

5 Performance

To evaluate the effectiveness and efficiency of our SSH honeypot system, we establish key performance indicators (KPIs) focusing on attack detection accuracy, system performance, deception effectiveness, and data analysis capabilities. These measures ensure that the honeypot remains undetectable while capturing valuable cyber threat intelligence.

5.1. Intrusion Detection & Logging Efficiency To assess how effectively the honeypot captures and logs attacker activities:

• Attack Detection Rate: Successfully identifying at least 95% of intrusion attempts.
• Log Completeness: Ensuring that 100% of login attempts and commands are recorded.
• Missed Attacks: Keeping undetected intrusion attempts below 5%.
• Time to Log Events: Logging attack data within 500ms for real-time monitoring.

5.2 Machine Learning Accuracy & Threat Classification • To measure the system's ability to analyse and classify attack behaviours:

• Anomaly Detection Accuracy: Achieving at least 90% accuracy in identifying suspicious activity.
• False Positive Rate: Keeping incorrect threat classifications below 5%

• Attack Pattern Recognition: Correctly clustering at least 85% of command sequences using TF-IDF and Isolation Forest.
• Real-Time Behaviour Analysis: Processing attacker commands and updating ML models within 2 seconds.

5.3 System Performance & Scalability To maintain a stable and high-performing honeypot environment:

• System Uptime: Ensuring 99.9% availability for uninterrupted attack logging.
• Latency: Keep ping honeypot response times under 1 second for seamless interaction with attackers.
• Log Storage & Retrieval Time: Retrieving stored attack logs within 300ms for realtime analysis.
• Concurrent Session Handling: Supporting at least 100 simultaneous attacker sessions without performance degradation.

5.4 Deception Effectiveness & Honeypot Undetectability To ensure the honeypot remains disguised as a real system:

• Fingerprinting Resistance: Preventing detection by tools like NMAP and Shodan in at least 95% of scans.
• Session Duration: Keeping attackers engaged for at least 10 minutes per session to collect more data.
• Credential Success Rate: Using realistic fake credentials that attackers attempt to use at least 80% of the time.
• Command Interaction Complexity: Ensuring attackers execute a variety of commands, not just automated scripts.

5.5 Data Analysis & Visualization Effectiveness To measure how well attack data is processed and presented:

• Visualization Clarity: Providing graphical attack trends with at least 90% accuracy in representing data.

• Anomaly Trend Insights: Detecting and categorizing new attack techniques within 24 hours.

• Report Generation Speed: Producing detailed security reports within 5 seconds.

• Threat Intelligence Sharing: Ensuring processed data can be exported for cybersecurity research and forensic analysis.

---

6. ADVANTAGES

6.1 Smart Threat Detection with Machine Learning

• Uses Isolation Forest to detect unusual login attempts and brute force attacks.
• Learns from new attack data over time, improving accuracy.
• Reduces the risk of undetected threats by identifying suspicious activities early.

6.2 Detailed Attack Data Collection

• Logs IP addresses, login attempts, command sequences, and timestamps.
• Provides in-depth insights into attacker behaviour for forensic analysis.
• Captures more detailed data compared to basic honeypots like Cowrie.

6.3 Easy-to-Understand Visual Reports

• Uses bar charts, word clouds, and graphs to display attack patterns.
• Helps security analysts quickly identify common attack techniques.
• Makes it easier to spot unusual access routes and frequently used commands.

6.4 Adapts to New Attack Methods

• Continuously updates itself with new attack patterns.
• Detects emerging threats like credential stuffing and automated scripts.
• Evolves with the changing cybersecurity landscape for long-term protection.

6.5 Efficient and Scalable

• Works on low-cost hardware or virtual machines with minimal resources.

- Supports multi-threading to handle multiple SSH connections at once.
- Ideal for large-scale security monitoring while keeping costs low.

6.6 Highly Accurate Detection

- Uses machine learning to distinguish between real threats and normal activity.
- Reduces false positives, ensuring legitimate users are not mistakenly flagged.
- Improves over time by adjusting detection thresholds based on data.

6.7 Proactive Security Measures

- Detects and classifies attacks in real-time, providing early warnings.
- Enables organizations to respond quickly to suspicious activity.
- Goes beyond passive monitoring by actively identifying attack patterns.

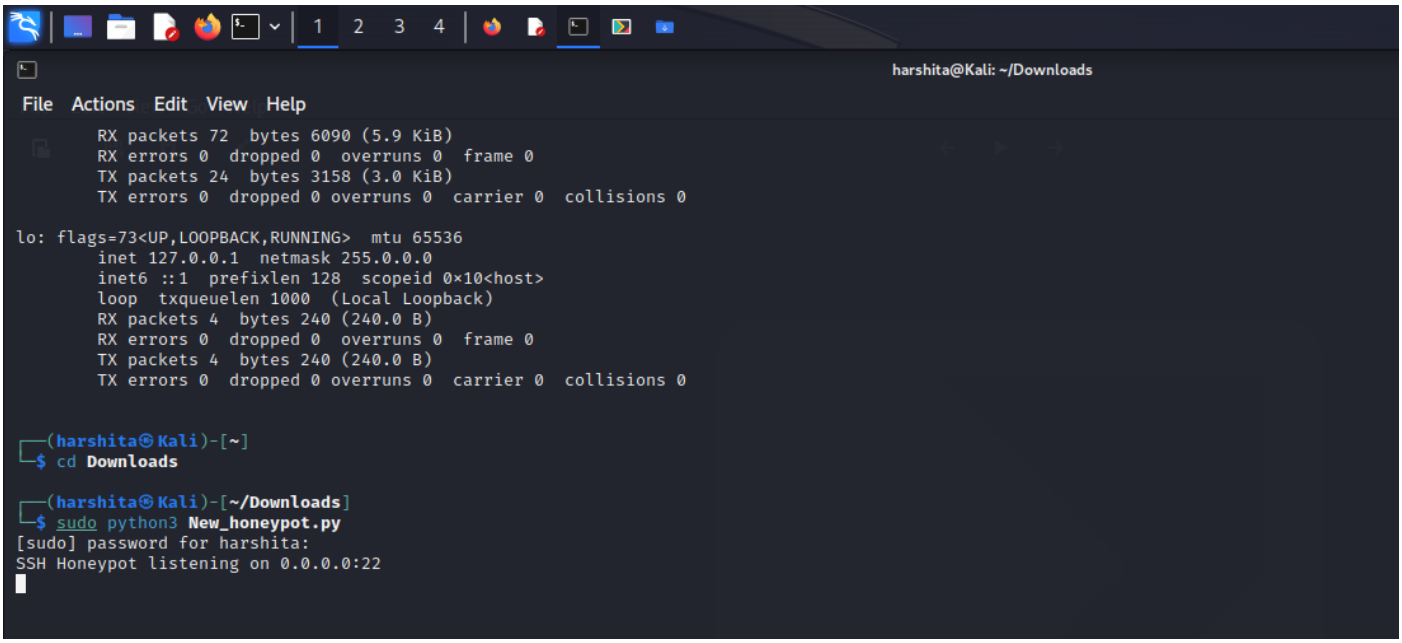
6.8. Automated and User-Friendly

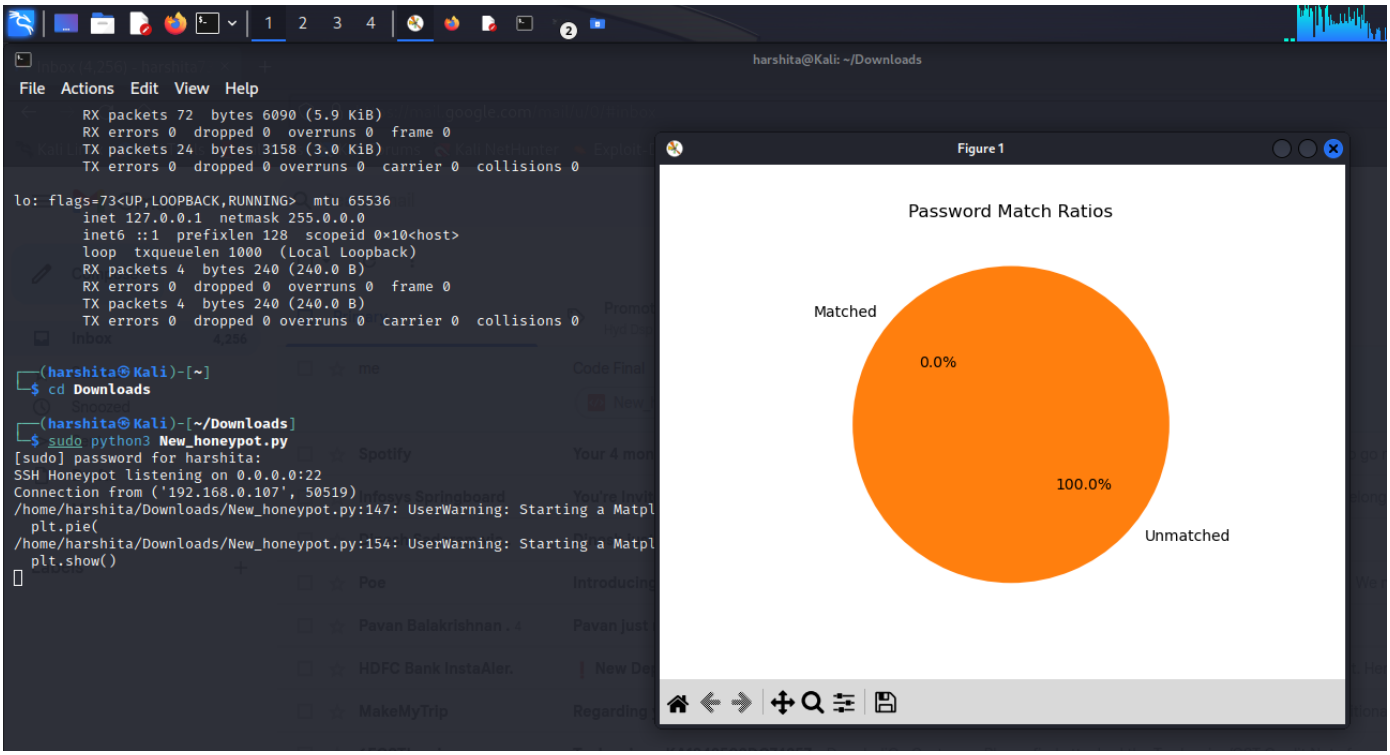
- Uses regular expressions (regex) to automatically clean and process logs.
- Transforms raw data into structured formats for easy machine learning analysis.
- Can be integrated with SIEM platforms for real-time security monitoring.
- This system makes cybersecurity monitoring smarter, faster, and easier while providing detailed insights into hacker behaviour.

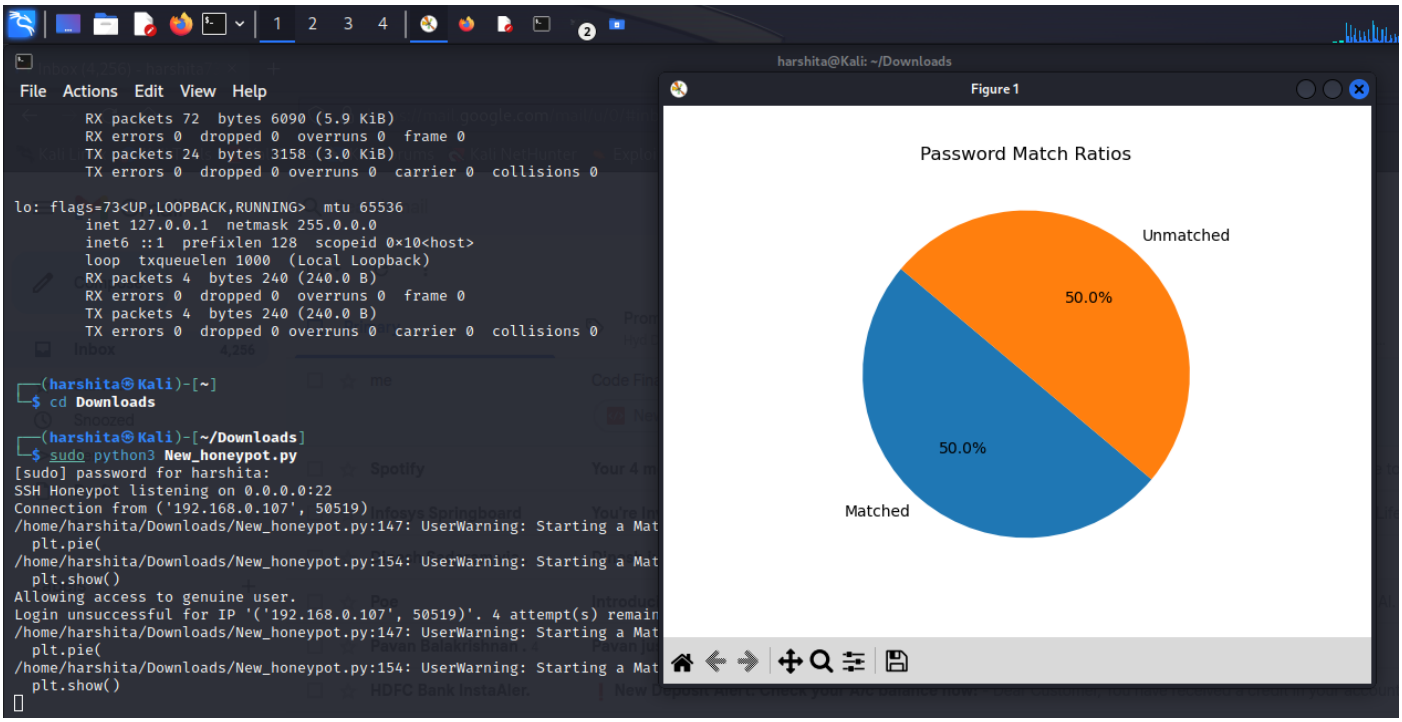
---

7. OUTPUTS:

Honeypot script running on Kali Linux




harshita@Kali: ~/Downloads
File Actions Edit View Help
RX packets 72 bytes 6090 (5.9 KiB)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 24 bytes 3158 (3.0 KiB)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
inet 127.0.0.1 netmask 255.0.0.0
inet6 ::1 prefixlen 128 scopeid 0x10<host>
loop txqueuelen 1000 (Local Loopback)
RX packets 4 bytes 240 (240.0 B)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 4 bytes 240 (240.0 B)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
(harshita㉿Kali)-[~]
$ cd Downloads
(harshita㉿Kali)-[~/Downloads]
$ sudo python3 New_honeypot.py
[sudo] password for harshita:
SSH Honeypot listening on 0.0.0.0:22
Connection from ('192.168.0.107', 50519)
/home/harshita/Downloads/New_honeypot.py:147: UserWarning: Starting a Matpl
plt.pie(
/home/harshita/Downloads/New_honeypot.py:154: UserWarning: Starting a Matpl
plt.show()
Figure 1
Password Match Ratios
Matched
0.0%
100.0%
Unmatched


harshita@Kali: ~/Downloads
File Actions Edit View Help
RX packets 72 bytes 6090 (5.9 KiB)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 24 bytes 3158 (3.0 KiB)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
inet 127.0.0.1 netmask 255.0.0.0
inet6 ::1 prefixlen 128 scopeid 0x10<host>
loop txqueuelen 1000 (Local Loopback)
RX packets 4 bytes 240 (240.0 B)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 4 bytes 240 (240.0 B)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
(harshita㉿Kali)-[~]
$ cd Downloads
(harshita㉿Kali)-[~/Downloads]
$ sudo python3 New_honeypot.py
[sudo] password for harshita:
SSH Honeypot listening on 0.0.0.0:22
Connection from ('192.168.0.107', 50519)
/home/harshita/Downloads/New_honeypot.py:147: UserWarning: Starting a Mat
plt.pie(
/home/harshita/Downloads/New_honeypot.py:154: UserWarning: Starting a Mat
plt.show()
Allowing access to genuine user.
Login unsuccessful for IP '('192.168.0.107', 50519)'. 4 attempt(s) remain
/home/harshita/Downloads/New_honeypot.py:147: UserWarning: Starting a Mat
plt.pie(
/home/harshita/Downloads/New_honeypot.py:154: UserWarning: Starting a Mat
plt.show()
Figure 1
Password Match Ratios
Unmatched
50.0%
50.0%
Matched

```
harshita@Kali: ~/Downloads
File  Actions  Edit  View  Help
        RX packets 72  bytes 6090 (5.9 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 24  bytes 3158 (3.0 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 4  bytes 240 (240.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 4  bytes 240 (240.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

┌──(harshita㉿Kali)-[~]
└─$ cd Downloads
┌──(harshita㉿Kali)-[~/Downloads]
└─$ sudo python3 New_honeypot.py
[sudo] password for harshita:
SSH Honeypot listening on 0.0.0.0:22
Connection from ('192.168.0.107', 50519)
/home/harshita/Downloads/New_honeypot.py:147: UserWarning: Starting a Matplotlib GUI outside of the main thread will likely fail.
  plt.pie(
/home/harshita/Downloads/New_honeypot.py:154: UserWarning: Starting a Matplotlib GUI outside of the main thread will likely fail.
  plt.show()
Allowing access to genuine user.
Login unsuccessful for IP '('192.168.0.107', 50519)'. 4 attempt(s) remaining before access is granted.
/home/harshita/Downloads/New_honeypot.py:147: UserWarning: Starting a Matplotlib GUI outside of the main thread will likely fail.
  plt.pie(
/home/harshita/Downloads/New_honeypot.py:154: UserWarning: Starting a Matplotlib GUI outside of the main thread will likely fail.
  plt.show()
Allowing access to genuine user.
Login unsuccessful for IP '('192.168.0.107', 50519)'. 3 attempt(s) remaining before access is granted.
```

On Windows Machine, when user is trying to connect to the honeypot

```
Administrator: Command Prompt - ssh  user@192.168.0.118
Microsoft Windows [Version 10.0.26100.3194]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\System32>ssh user@192.168.0.118
user@192.168.0.118's password:
Permission denied, please try again.
user@192.168.0.118's password:
Permission denied, please try again.
user@192.168.0.118's password:
Permission denied, please try again.
user@192.168.0.118's password:
Permission denied, please try again.
user@192.168.0.118's password:
```

```
Administrator: Command Prompt

/ $ ls
root home etc var

/ $ cd home
/home $ ls
user admin

/home $ cd ..
/ $ pwd
/

/ $ cd root
/root $ ls
bin boot dev lib usr

/root $ cd ..
/ $ cd etc
/etc $ ls
shadow hostname hosts username passwd

/etc $ cat username
['admin_user: admin123', 'test_user: testpass']

/etc $ cat passwd
['root: P@ssw0rd', 'admin: 123456']

/etc $ cd ..
/ $ exit
Goodbye!
Connection to 192.168.0.118 closed by remote host.
Connection to 192.168.0.118 closed.
```

8.DRAWBACKS

• Limited Attack Interaction: As a low-interaction honeypot, it may not fully capture complex, multi-step attack scenarios or advanced persistent threats (APTs).

• Accuracy Concerns: While machine learning enhances detection, anomalies in attacker behaviour may lead to false positives or missed threats.

• Resource Intensity: Running machine learning models for real-time analysis requires significant computational power, which may limit deployment on low-resource systems.

• Potential Detection by Attackers: Skilled attackers may recognize the honeypot environment, avoiding interaction or using evasion techniques to bypass logging.

• Data Privacy Challenges: Storing and analysing attacker logs involves handling sensitive data, requiring strict security measures to prevent misuse or leaks.

• No Machine Learning: It does not use ML models to detect patterns or predict behaviours based on logs.

• Command Analysis: Primarily focused on malware rather than interactive attacker behaviour like SSH commands

9. CONCLUSION

Our SSH honeypot system provides an advanced cybersecurity solution by combining machine learning, real-time monitoring, and detailed attack analysis. Unlike traditional honeypots, which rely solely on static logging, this system enhances threat detection, attacker behaviour analysis, and proactive security measures. By leveraging Isolation Forest for anomaly detection, TF-IDF for attack pattern recognition, and graph theory for command flow visualization, the honeypot efficiently identifies suspicious activities and provides meaningful insights into evolving cyber threats.

The system's adaptive learning ensures it continuously improves, making it highly effective against emerging attack strategies like credential stuffing, automated scripts, and brute force attacks. Additionally, the system is designed for scalability and efficiency, capable of running on minimal hardware resources while supporting large-scale deployments. Its automated log parsing, real-time alerts, and SIEM integration make it a user-friendly and proactive cybersecurity tool for organizations seeking to strengthen their network defences. Overall, this project enhances cybersecurity by providing real-time threat intelligence, improving attack detection accuracy, and reducing response time to security incidents. By continuously evolving, the system remains a powerful tool for preventing, detecting, and analysing SSH-based attacks in modern cybersecurity landscapes.

REFERENCES

1. https://www.researchgate.net/publication/368921265_Analysis_of_SSH_Honeypot_Effectiveness
2. https://ieeexplore.ieee.org/document/6624967
3. https://dl.acm.org/doi/10.1145/3617184.3618056?utm_source=chatgpt.com
4. https://ieeexplore.ieee.org/abstract/document/9558948
5. https://link.springer.com/chapter/10.1007/978-3-319-18467-8_18?fromPaywallRec=true
6. https://medium.com/acmvit/ssh-honeypot-build-your-own-6f508d535672
7. https://ieeexplore.ieee.org/document/8888409
8. https://ieeexplore.ieee.org/document/9633881
9. https://lwn.net/Articles/848291/
10. https://inria.hal.science/hal-03746028/file/512098_1_En_21_Chapter.pdf